

AI Level Director Studio: A Designer Governed Workflow for 2D Platformer Level Iteration

Student: Robert Mayfield

Industry Context and Problem Definition

AI Level Director Studio targets the creative industries, specifically independent and small team game development, a sector reshaped by the pressures that make responsible AI integration urgent. The Game Developers Conference 2025 State of the Game Industry survey, drawn from more than 3,000 developers, found that 11% had been laid off in the past year, while 52% worked at companies that had implemented generative AI. Adoption is rising, but confidence is not: 30% believed generative AI was having a negative impact on the industry, up 12 percentage points over the prior year, and they named intellectual property theft, falling quality, bias, and regulation among their concerns (Game Developers Conference, 2025).

That combination, more generated content, fewer people, and falling trust, is the real problem to design for. Level design is itself a blend of science and craft. Structural qualities such as difficulty, pacing, and a safe opening can be measured and informed by established design patterns, yet whether a segment is genuinely engaging, fun, and fairly challenging is a creative judgment that rests on a designer's experience as well as an iterative design process. Mixed initiative tools have long treated level design as a collaboration between designer and system rather than full automation (Smith et al., 2010), so it supports the designer rather than replaces them. Review and playtesting are expensive and leave a small team little slack. Generative AI can produce candidate content quickly, a reasonable tool for early iteration, but the harder problem is not generation: raw generated content cannot be trusted as a finished asset, because it may be structurally invalid, misaligned with the design intent, or derivative of its training data.

The need is therefore the move from “an AI can produce a candidate” to “a designer can responsibly decide what to do with that candidate.” This is an applied system design problem under real constraints: a small team needs something lightweight and transparent, and originality and intellectual property risk must be disclosed rather than hidden. AI Level Director Studio addresses that gap. It helps a designer screen, triage, and iterate on candidate level segments faster while keeping every creative decision in human hands.

Overview of the Integrated Solution

AI Level Director Studio is a designer governed workflow for iterating on 2D platformer level segments. Rather than a new model, the system is a compound AI system, reflecting the observation that many effective AI applications combine models, tools, and orchestration rather than rely on one monolithic model (Zaharia et al., 2024). AI Level Director Studio contributes the orchestration layer that turns three prior AI projects into one coherent loop, together with the candidate lifecycle management, persistence, reporting, and interface that make the loop usable and transparent.

The workflow runs from a brief to generated or uploaded candidates, agentic triage, a playtest decision, feedback classification, and finally completing or revising the candidate. Every candidate carries its own lifecycle state, so several can sit at different stages within one session.

The system is deliberately advisory. It recommends and routes, but it never edits a level, never finalizes one on its own, and never claims to predict whether a level is fun. Its outputs are observable and reviewable: a candidate board, triage results, classified feedback, JSON session snapshots, an append only event log, and a Markdown report. The same backend drives a reproducible notebook and a Gradio interface, so the workflow can be demonstrated or rerun (Figure 1).

Integration of Prior Projects and Methods

The system integrates three of my prior capstone projects, each wrapped behind an adapter so the studio coordinates them without rewriting them. In each case a lesson from the original project shaped its role here.

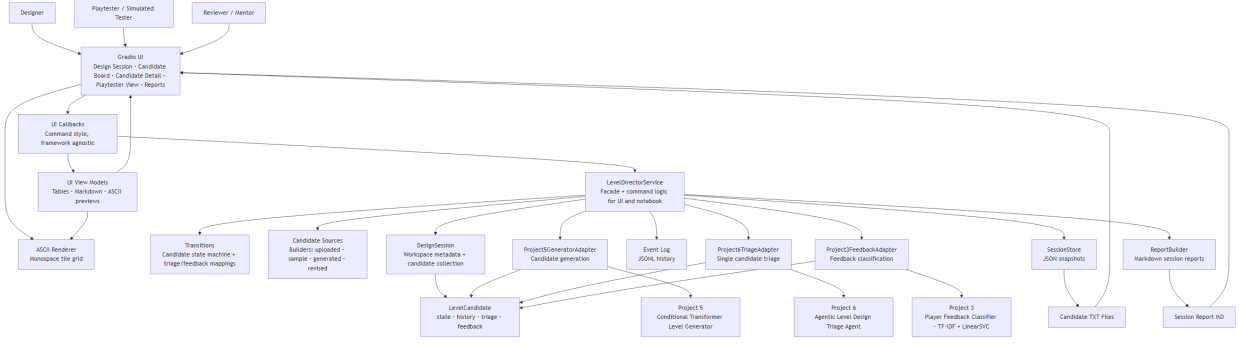


Figure 1: System architecture. The Gradio UI and the reproducible notebook call one LevelDirectorService facade, which delegates lifecycle rules to the state machine and reaches the Generative AI, Agentic AI, and Machine Learning projects only through adapters. Every step persists as a JSON session snapshot and a JSONL event log.

1. **Generative AI, the candidate source.** The conditional Transformer level generator proposes candidate segments. Procedural content generation via machine learning is valuable not only for autonomous generation but for co-creation, critique, and content analysis, and is constrained by the scarcity of training data for customized game content (Summerville et al., 2018). My results showed this directly: the generator produced structurally valid, difficulty controlled segments, but novelty analysis revealed substantial similarity to the training levels. The studio therefore treats generated levels as drafts and discloses their derivative risk rather than hiding it.
2. **Agentic AI, the triage authority.** The level design triage agent is the design judgment layer and the single triage authority. It interprets a brief, gathers facts with deterministic tools, reasons about how a candidate serves the intent, and recommends one of six actions, following the pattern in which a model interleaves reasoning with tool grounded actions rather than relying on internal knowledge alone (Yao et al., 2022). The lesson I carried forward is that the agent’s value is judgment, not facts, so it is reused unchanged and its reasoning is never duplicated.
3. **Machine Learning, the feedback signal.** The player feedback classifier supplies the post playtest reception signal, classifying playtester text as positive or negative. Because a sentiment classifier is useful but narrow, it runs only after a candidate is sent to playtest and is treated as a broad signal, not a complete playtest analysis.

In sequence, Generative AI proposes, Agentic AI judges, and Machine Learning reports back, while the studio owns the lifecycle, persistence, and reporting that connect them. Three earlier projects inform the framing: Programming Foundations reinforced data scope caution, Statistical Analysis showed that a statistically significant signal can still have weak practical value, and Deep Learning is reserved as a future live telemetry extension.

Technical Design Decisions and Tradeoffs

The decisions that shaped the system were about its overall design and what to integrate, each weighing a real alternative.

Decision A: A designer governed loop, not an autonomous engine. A pipeline that generates and finalizes levels on its own was rejected because level design is part craft and the designer must keep creative authority. The system advises and routes; it never decides. This is the spine of the synthesis.

Decision B: Treat every signal as evidence, never a verdict. The alternative, a single score that approves candidates, fails because no model can judge whether a level is fun, and a small studio has little reason to trust a black box that claims to. Validity, difficulty, novelty, pacing, and sentiment are evidence for the designer to weigh, consistent with human AI interaction guidelines that stress user control and clear limits (Amershi et al., 2019).

Decision C: A designer facing UI, not a notebook only demonstration. A notebook would prove the pipeline runs, but a UI makes the independent but connected nature of the components tangible: a designer drives the loop and sees each system’s distinct contribution, which is what builds trust in an AI assisted tool.

Decision D: Non-destructive, fully audited iteration. Rather than one mutable level, each candidate owns its state, revisions create new linked candidates, and every action appends to an event log (Figure 2). Overwriting as you go is simpler but loses the variant history designers iterate on and the provenance accountability needs.

Decision E: Composable, swappable components behind adapters. Wiring the three systems together directly invites the entanglement that makes machine learning systems fragile (Sculley et al., 2015). Adapters let a small team upgrade or replace one model without rebuilding the system, keeping the orchestration dependent on stable interfaces, not model internals.

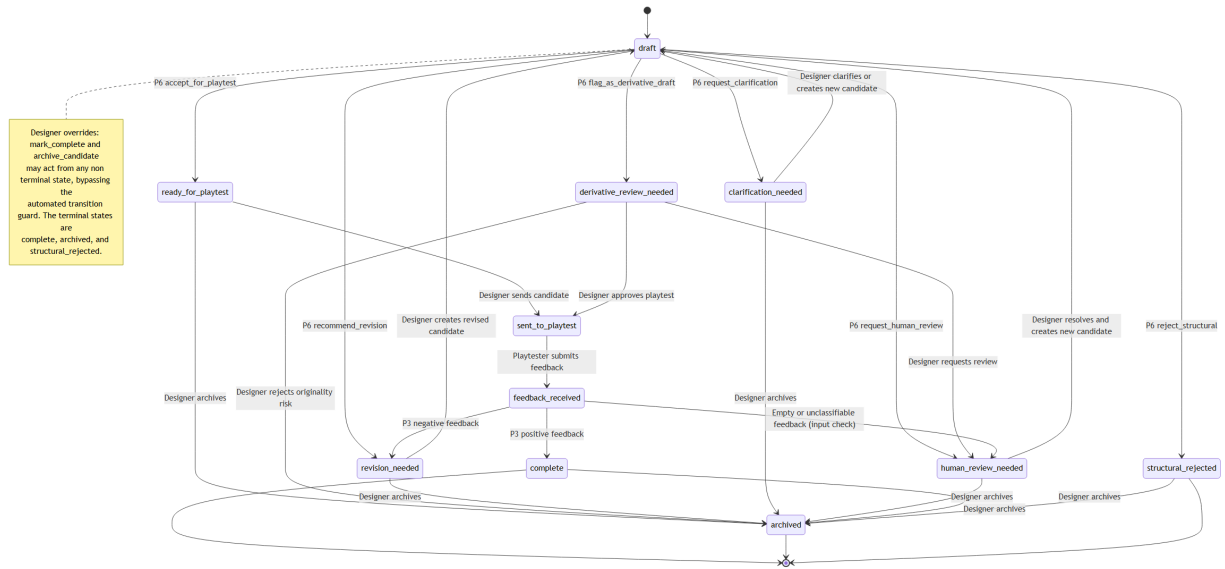


Figure 2: Candidate lifecycle state machine. Triage and feedback drive the automated transitions, while the designer can complete or archive a candidate from any non terminal state. Each candidate owns its own state, so several candidates at different stages coexist in one session.

Evaluation and Reflection

Across scenarios A through E the loop behaved as designed: a generated candidate was screened and flagged for derivative risk, an ambiguous brief drew a clarification request rather than a forced recommendation, and feedback completed or revised a candidate. The designer governed loop held, routing evidence to the human and never finalizing a level itself, which shows sensible behavior, not improved design outcomes.

The harder work was in the seams, not the models. The three systems shared no common notion of a candidate, and each returned a different kind of output, raw tile text, a structured action, and a bare sentiment label, with different certainty semantics. Reconciling those into one advisory lifecycle, and routing weak or conflicting signals without letting any one dominate, was harder than running any single model. Because the triage authority is a language model, the same candidate can draw different actions on different runs, a real tension with a reviewable workflow. The lesson is that integrating across domains pays off in the orchestration and the boundaries, not the modeling.

Ethical, Governance, and Responsible AI Considerations

Derivative content is the clearest risk. The generator was trained on a small corpus and its outputs showed high similarity to it, so a generated candidate can closely resemble an existing level, an intellectual property risk in a commercial setting and one developers themselves flagged (Game Developers Conference, 2025). The studio addresses it directly: generated levels are labeled as drafts, their derivative risk is surfaced as a warning rather than hidden, and a candidate that is too similar can be routed to human originality review instead of being accepted automatically.

Over trust, treating a model’s output as a verdict, is the second risk. The feedback classifier is a broad reception signal, not a complete playtest analysis, and the triage agent offers judgment, not ground truth about whether a level is fun. The system keeps a human in control and communicates uncertainty, in line with human AI interaction guidelines emphasizing transparency, user control, and clear limits (Amershi et al., 2019): it never edits or finalizes a level, treats every signal as evidence, supports clarification and human review states, saves the agent’s full reasoning transcript for each triage so any recommendation can be audited, and discloses its limitations in every report (Mitchell et al., 2019).

Responsible deployment requires more than the prototype provides: broader training data, far more playtesting, asset license review, privacy handling for real feedback, and professional quality assurance. Accountability stays with the human designer.

Limitations and Risks

This is a prototype, and its limits are real.

- **Generator:** trained on a small corpus, so its candidates carry derivative risk and are not production ready content.
- **Classifier:** trained on game review text, so it generalizes only loosely to short playtest notes, and it returns a label with no calibrated confidence.
- **Triage agent:** its difficulty estimate is heuristic and structural, not player validated, and because it uses a language model its judgments vary between runs and depend on an external provider.
- **Playtesting:** simulated rather than connected to real players.

The main standing risk is over reliance: if a team treated these advisory signals as final decisions, the system’s value would invert. The designer governed design mitigates that risk but does not remove it.

Professional and Industry Relevance

The pressing question in game development today is not whether AI can generate content, it clearly can, but whether a studio can adopt it without sacrificing originality, quality, or the trust of its team. AI Level Director Studio is a small, working answer. It treats AI as designer governed assistance that accelerates iteration while keeping creative authority, transparency, and accountability with the human: generated content is screened, not shipped; every signal is evidence, not a verdict; and the full history of decisions is recorded for review. Integrating specialized models into an accountable workflow, rather than pursuing a system that designs on its own, is what studios increasingly need as they adopt AI, and the pattern generalizes to any creative or high stakes field bringing it in responsibly. Showing that an integrated system can speed a designer up without taking the design out of their hands is the contribution that matters most.

Future Extensions or Improvements

Several improvements follow from the limitations above.

1. **Calibrated confidence on the classifier**, for example through Platt scaling, so the system could express how certain a feedback judgment is and route uncertain cases to human review instead of trusting every label equally.
2. **A stronger generator** that reduces derivative risk, for instance a larger pretrained model rather than a small from scratch one.

3. **Live telemetry from the deep learning project**, so real player behavior signals complement post playtest sentiment, and connecting the playtester step to real players to enable proper user studies of whether the workflow improves design outcomes.
4. **Lighter polish**: image or sprite based previews using appropriately licensed assets, and an optional report narrator that narrates around the ground truth facts rather than generating them.

References

- Amershi, S., Weld, D., Vorvoreanu, M., Fourney, A., Nushi, B., Collisson, P., Suh, J., Iqbal, S., Bennett, P. N., Inkpen, K., Teevan, J., Kikin-Gil, R., & Horvitz, E. (2019). Guidelines for human-AI interaction. *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, 1-13. <https://doi.org/10.1145/3290605.3300233>
- Game Developers Conference. (2025). *GDC 2025 state of the game industry: Devs weigh in on layoffs, AI, and more*. <https://gdconf.com/article/gdc-2025-state-of-the-game-industry-devs-weigh-in-on-layoffs-ai-and-more/>
- Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., & Gebru, T. (2019). Model cards for model reporting. *Proceedings of the Conference on Fairness, Accountability, and Transparency*, 220-229. <https://doi.org/10.1145/3287560.3287596>
- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems*, 28. <https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems.pdf>
- Smith, G., Whitehead, J., & Mateas, M. (2010). Tanagra: A mixed-initiative level design tool. *Proceedings of the Fifth International Conference on the Foundations of Digital Games*, 209-216. <https://doi.org/10.1145/1822348.1822376>
- Summerville, A., Snodgrass, S., Guzdial, M., Holmgård, C., Hoover, A. K., Isaksen, A., Nealen, A., & Togelius, J. (2018). Procedural content generation via machine learning (PCGML). *IEEE Transactions on Games*, 10(3), 257-270. <https://doi.org/10.1109/TG.2018.2846639>
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). *ReAct: Synergizing reasoning and acting in language models*. <https://arxiv.org/abs/2210.03629>
- Zaharia, M., Khattab, O., Chen, L., Davis, J. Q., Miller, H., Potts, C., Zou, J., Carbin, M., Frankle, J., Rao, N., & Ghodsi, A. (2024). *The shift from models to compound AI systems*. Berkeley Artificial Intelligence Research. <https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/>